

Top 10 web hacking techniques of 2019



James Kettle

Director of Research

[@albinowax](#)



 **Published:** Monday, 17 February 2020 at 14:36 UTC

Updated: Tuesday, 5 January 2021 at 14:10 UTC



The results are in!

After [51 nominations](#) whittled down to 15 finalists by a community vote, an expert panel consisting of [Nicolas Grégoire](#), [Soroush Dalili](#), [Filedescriptor](#), and [myself](#) have conferred, voted, and selected the Top 10 new web hacking techniques of 2019.

Every year, professional researchers, seasoned pentesters, bug bounty hunters and academics release a flood of blog posts, presentations, videos and whitepapers. Whether they're suggesting new attack techniques, remixing old ones, or documenting findings, many of these contain novel ideas that can be applied elsewhere.

However, in these days of vulnerabilities arriving equipped with logos and marketing teams it's all too easy for innovative techniques and ideas to get missed in the noise, simply because they weren't broadcast loudly enough. That's why every year, we work with the community to seek out and enshrine ten techniques that we think will withstand the test of time.

We regard these ten as the creme of the most innovative web security research published in the last year. Every entry contains insights for aspiring researchers, pentesters, bug bounty hunters, and anyone else interested in recent developments in web security.

Community Favourite - HTTP Desync Attacks

The entry with the most community votes by a substantial margin was [HTTP Desync Attacks](#), in which I revived the long forgotten technique of [HTTP Request Smuggling](#) to earn over \$90k in bug bounties, compromise PayPal's login page twice, and kick off a wave of findings for the wider community. I regard this as my best research to date, but I made the tactical decision to exclude it from the official top 10 because there's no way I'm going to write a post that declares my own research the best. Moving swiftly on...

10. Exploiting Null Byte Buffer Overflow for a \$40,000 bounty

At number 10 we have a fantastic heartbleed-style [memory-safety exploit](#) from [Sam Curry](#) and friends. This critical but easily-overlooked vulnerability almost certainly affects other websites, and serves us a reminder that even if you're an expert, there's still a place for simply fuzzing and keeping an eye out for anything unexpected.

9. Microsoft Edge (Chromium) - EoP to Potential RCE

In this writeup, [Abdulrhman Alqabandi](#) uses a mixture of [web and binary attacks](#) to pwn anyone who makes the mistake of visiting his site using Microsoft's new Chromium-Powered Edge (aka Edgium).

\$40,000 in bounties later this is now patched, but it's still a sterling example of an exploit chain combining multiple low-severity vulnerabilities to achieve a critical impact, and also beautifully demonstrates how web vulnerabilities can bleed onto your desktop through privileged origins. It inspired us to update [Hackability](#) to detect when it's on a privileged origin by scanning the chrome object.

For another look at web vulnerability chaos in the browser-chrome battleground, check out [Remote Code Execution in Firefox beyond memory corruptions](#).

8. Infiltrating Corporate Intranet Like NSA: Pre-Auth RCE On Leading SSL VPNs

The incumbent winner [Orange Tsai](#) makes his first appearance alongside [Meh Chang](#) with multiple unauthenticated RCE [vulnerabilities in SSL VPNs](#).

The privileged, internet-exposed position VPNs typically sit in means that in terms of sheer impact, this is about as good as it gets. Although the techniques applied are largely classics, they use some creative twists that I won't spoil for you here. This research helped spawn a wave of audits targeting SSL VPNs, leading to numerous findings including a [clutch of SonicWall vulnerabilities](#) published last week.

7. Exploring CI Services as a Bug Bounty Hunter

Modern websites are stitched together from numerous services reliant on secrets to identify each-other. When these get leaked, the web of trust can fall apart. Secrets leaking in Continuous Integration repositories/logs is a common occurrence, and finding them via automation is even more common. Yet [this research](#) by [EdOverflow](#) et al systematically sheds new light on overlooked cases and potential future research areas. It's also quite possibly the inspiration for the hilarious site/tool [SSHGit](#).

6. All is XSS that comes to the .NET

Monitoring novel research is a core part of my job, but I still managed to completely miss this post when it was first released. Fortunately, someone in the community had sharper eyes and nominated it.

[Paweł Hądrzyński](#) takes a [little-known legacy feature](#) of the .NET framework and shows how it can be used to add arbitrary content to URL paths on arbitrary endpoints, causing us some mild panic when we realised even our own website supported it.

Reminiscent of [Relative Path Overwrite](#) attacks, this is a piece of arcana that can sometimes kick off an exploit chain. In the post it's used for [XSS](#), but we strongly suspect alternative abuses will emerge in future.

5. Google Search XSS

The Google Search box is probably the most-tested input on the planet, so how [Masato Kinugawa](#) managed to XSS it was beyond comprehension, up until he revealed all via a collaboration with his colleague [LiveOverflow](#).

These two videos provide a solid introduction on how to [find DOM parsing bugs](#) by reading the docs and fuzzing, and also give a [rare look into](#) the creativity behind this magnificent exploit.

4. Abusing Meta Programming for Unauthenticated RCE

Orange Tsai returns with a pre-auth RCE in Jenkins, described over two posts. The [authentication bypass](#) is nice, but our favourite innovation is the [use of meta-programming](#) to create a backdoor that executes at compile-time, in the face of numerous environmental constraints. We expect to see meta-programming again in future.

It's also an excellent example of research continuation, as the exploit was subsequently [improved by multiple researchers](#).

3. Owing The Clout Through Server Side Request Forgery

This [presentation](#) from [Ben Sadeghipour](#) and [Cody Brocius](#) starts out with an overview of existing [SSRF](#) techniques, shows how they can be adapted and applied to server-side PDF generators, then brings DNS rebinding into the mix for good measure.

The work targeting PDF generators is an insightful look into a feature-class that's all too easily ignored. We first saw [DNS rebinding on server-side browsers](#) appear on the [2018 nomination list](#), and the release of HTTPRebind should help make this attack more accessible than ever.

Finally, I might be wrong about this but I suspect this presentation may deserve some credit for finally persuading Amazon to think about [securing their EC2 metadata endpoint](#).

2. Cross-Site Leaks

Cross-site leaks have been a long time coming. First documented [over a decade ago](#), and creeping into our [top 10 last year](#), it's in 2019 that awareness of this attack class and its sheer number of crazy variations exploded.

It's hard to apportion credit at such a scale but we clearly owe thanks to [Eduardo Vela's succinct introduction](#) to the concept with a novel technique, the collaborative effort to build a public [list of known XS-Leak vectors](#), and researchers applying the XS-Leaks technique to [great effect](#).

XS-Leaks have already had a lasting impact on the web security landscape, as they played a major role in the death of browser XSS filters. Block-mode XSS filtering was a major source of XS-Leak vectors, and this combined with [even worse issues with filter-mode](#) to persuade Edge and later Chrome to both discard their filters in a victory for web security and a disaster for web security researchers alike.

1. Cached and Confused: Web Cache Deception in the Wild

In [this academic whitepaper](#), [Sajjad Arshad](#) et al take Omer Gil's [Web Cache Deception](#) technique (which premiered at #2 in our top 10 back in 2017), and share a systematic exploration of [Web Cache Deception](#)

vulnerabilities across the Alexa Top 5000 websites.

For legal reasons, most offensive security research is conducted during professional audits or on websites with bug bounty programs, but through careful ethical footwork this research offers a glimpse into the state of security on the wider web. With the help of a well-crafted methodology that could easily be adapted for other techniques, they prove that Web Cache Deception is still a prevalent threat.

Aside from the methodology, the other key innovation is the introduction of five novel path confusion techniques which expand the number of vulnerable websites. They also do a better job of documenting web-caching provider's caching behaviour than many providers themselves. Overall, this is a superb example of the community taking existing research in a new direction, and a well deserved number one!

Conclusion

We saw a particularly strong set of nominations this year, so many excellent pieces of research didn't make it into the top 10. As such, I recommend checking out the [full nomination list](#). For those interested in getting access to 2020 research as soon as it's released, we recently created the [r/websecurityresearch](#) subreddit and [@PortSwiggerRes](#) Twitter accounts to promote notable research. You can also find past year's top 10 lists here:

[2018](#), [2017](#), [2015](#), [2014](#), [2013](#), [2012](#), [2011](#), [2010](#), [2009](#), [2008](#), [2007](#), [2006](#).

Year after year we see great research comes from building on other people's ideas, so we'd like to thank everyone who takes the time to publish their findings, whether nominated or not. Finally, we'd like to thank the wider community for your enthusiastic participation. Without your nominations and votes, this wouldn't be possible.

Till next year!

[top-10-techniques](#)

[Top 10 Hacking Techniques](#)

[← Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering



© 2026 PortSwigger Ltd.

